

Libra 版本化研发过程记忆模块项目申报书

Libra 版本化研发过程记忆模块项目申报书

项目背景

项目基本需求

项目相关仓库

技术方法及可行性

总体架构

核心技术方法

版本关系

轻量化存储方案

可行性

项目实现细节梳理

1. 研发回合与需求迭代

2. 摘要数据契约

3. 自动编译流程

4. 召回与上下文组装

5. 内部模块

6. Benchmark 方案

项目开发规划

项目背景

Libra 是面向 AI Agent 的版本控制与协同追溯系统。当前项目已经记录需求意图（Intent）、执行计划（Plan）、研发任务（Task）、执行尝试（Run）、会话（Session）、工具调用（ToolInvocation）、执行证据（Evidence）、决策（Decision）、代码修改（PatchSet）和 Git 代码对象。这些对象保存了一次研发活动的原始过程。

随着 Agent 持续参与开发，后续任务需要快速理解三类信息：当前代码如何形成、过去尝试过哪些方案、历史结论与当前代码版本有什么关系。直接重放完整 Session 成本较高，也很难从多次 Run 中还原需求变化、方案取舍和失败原因。

本项目实现“版本化研发过程记忆”：系统沿用 Libra 现有研发对象，自动编译任务级与需求级摘要，并把摘要保存为带 Git 历史的长期记忆。Agent 先读取摘要，再按问题展开到 Run、Evidence、Session、PatchSet 和代码对象。这样可以回顾一次失败的根因、理解一段代码的设计来源，也可以比较新需求与旧版本中的相关事件。

项目完成后，Libra 将具备以下能力：

- 从需求到代码版本追踪完整研发过程；
- 自动整理成功、失败、重试、取消和未完成的 Run；
- 按 Task 汇总一次研发回合，按 Intent 汇总一次需求迭代；
- 将摘要与来源证据、代码对象标识（OID）和有效时间绑定；
- 根据当前任务和代码版本，为 Agent 组装可追溯的历史上下文。

项目基本需求

需求	项目定义	验收结果
完整研发链路	使用 Intent → Plan → Task → Run → Evidence / Decision / PatchSet → Git Commit，并关联 Session 与 ToolInvocation	任一摘要都能回到对应 Task、Run、证据和代码状态
全量过程记忆	Run 的完成、失败、取消、超时和无代码提交均进入同一研发历史	Agent 可以查询失败方案、未完成推断和后续重试
两级结构化摘要	研发回合摘要（Task Episode Summary）聚合一个 Task 的多个 Run；需求迭代摘要（Intent Iteration Summary）聚合一个 Intent 下的多个 Task	Agent 可以从需求概览逐层展开到具体执行过程
版本与时间绑定	摘要记录基础代码 OID、结果代码 OID、PatchSet、分支、相关路径和覆盖的事件范围	查询结果能说明历史结论对应哪个代码状态

事实与推断分栏	测试结果、工具输出和代码状态记为已观察事实；根因、设计理由和适用性记为 Agent 推断	每项主要结论均带来源引用和类型标记
自动运行	Run、Task 和 Intent 的生命周期事件驱动记忆编译	开发 Agent 完成任务后，由系统更新摘要
渐进读取	先读需求摘要，再读任务摘要，最后展开原始数据	在固定上下文预算内回答研发历史问题
初版召回	先按 Intent、Task、Run、代码版本、时间和路径过滤，再使用 SQLite 全文检索扩展（FTS5）与关键词相关性排序（BM25）检索两级摘要	无外部模型或检索服务时仍能稳定召回；同一冻结上下文的 Top K 结果可重放
轻量存储	权威摘要保存在 Libra 现有对象存储和受保护内部引用；现有 <code>.libra/libra.db</code> 增加可重建的 FTS5 投影，不引入新的数据库产品或常驻服务	本地单命令即可运行；摘要历史可 diff、追溯和恢复；删除全文索引后可以从 Memory ref 完全重建
可复现评测	建立来源、版本、回答质量、上下文成本和延迟评测	产生固定数据集、运行命令、JSON 结果与分析文章

研发回合（Development Episode）是以 Task 为根组装的读取结构。领域事实统一使用现有 Task、Run 和 Evidence；两级摘要作为编译产物更新，原始对象继续承担事实来源。

项目相关仓库

项目	具体工程实现	本项目采用的方法
Libra	Rust 单体 CLI；Git-compatible 对象存储；Intent、Task、Run、Evidence、Decision、PatchSet；Session 与 Agent runtime；内部引用与比较并交换（CAS）更新	作为领域模型、事实来源、存储系统和运行环境
Letta Code / MemFS	TypeScript CLI；用 Git 管理 Markdown 记忆文件；YAML 元数据头（frontmatter）保存机器字段；目录树常驻、正文按需读取；后台反思在独立 branch/worktree 中修改、提交并合并记忆	用 Markdown 表达可读摘要，用对象树组织 Task/Intent 文件，用独立记忆提交保留摘要修订历史
Rekal CLI / 论文	Go 单二进制 CLI；以两个本地 DuckDB 文件分别保存原始账本和可重建索引；提交后从不同 Agent 适配器读取 Session；记录 commit SHA、branch 和涉及文件（files touched）；索引组合 BM25、LSA 与本地 embedding；查询按当前代码、具体历史和设计理由分流	建立 Run/Session 与代码版本的来源链；按问题选择当前代码、任务历史或跨任务综合；从代码与会话关系构造 Benchmark，并用实测结果决定 Libra 的检索层
SQLite FTS Baseline	将原始消息同步写入 SQLite 普通表与 FTS5 索引；使用参数化 MATCH、BM25、Top K 和幂等请求提供无模型关键词检索	采用 FTS5/BM25 作为初版轻量召回基线；不采用其原始消息即记忆、无命中返回最近消息、英文分词器直接处理中文等做法

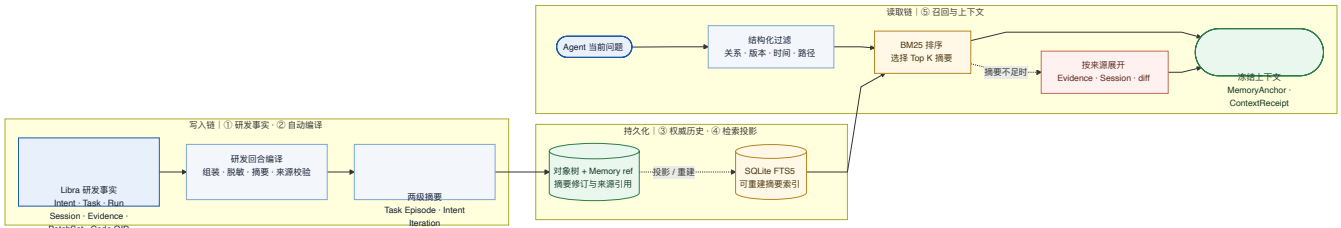
四个项目在本项目中的分工如下：

- Libra 提供“发生了什么”的权威研发对象；
- Rekal 提供“研发过程如何关联代码版本、如何查询”的工程参考；
- Letta 提供“摘要如何组织、更新和保留版本”的工程参考；
- SQLite FTS Baseline 提供“如何用现有 SQLite 建立确定性关键词检索下界”的工程参考。

Letta 的后台记忆分支与 Rekal 的提交后 checkpoint 都展示了自动整理流程。Libra 可以直接使用 Run、Task 和 Intent 的生命周期信号触发编译，获得比提交钩子更准确的任务边界。

技术方法及可行性

总体架构



图按写入、持久化、读取三栏呈现，逻辑仍分为五层：现有研发对象保存事实，记忆编译器生成摘要，受保护引用保存摘要版本，SQLite FTS5/BM25 提供可重建的关键词检索投影，召回器按问题组装上下文。摘要通过来源引用连接原始对象，代码内容和 diff 继续按 OID 从对象库读取。

核心技术方法

方法	实现方式	可行性依据
研发回合组装	以 <code>task_id</code> 聚合关联的 Run、Session、Evidence、Decision 和 PatchSet；用 <code>covered_event_range</code> 记录本次处理范围	Libra 已保存主要对象和生命周期信息
两级增量摘要	Task 终态生成研发回合摘要；Intent 修订或终态聚合相关 Task 摘要； <code>source_digest</code> 判断受影响文件	Letta 已验证 Git 文件格式记忆和增量提交路径
自动摘要 Agent	复用 Libra provider/runtime；输入为有界结构化数据；输出固定 schema；编译记录保存模型、提示词版本和来源摘要	现有 Agent runtime 可承载后台任务
Git-backed 存储	摘要写成 Markdown + YAML frontmatter，组成对象树；创建记忆版本提交（memory commit）；CAS 更新 <code>refs/libra/memory/process</code>	Libra 已具备对象、tree、commit、ref 与 CAS 基础
版本关系计算	结合当前代码 OID、摘要代码锚点、Git DAG、PatchSet 和相关路径，生成版本关系元数据	Libra 的 VCS 对象图可以直接计算 ancestry 与 diff
全文检索投影	将当前 Memory ref 中的 Task/Intent 摘要投影到现有 SQLite；FTS5 建立倒排索引，BM25 对关键词命中排序；索引记录绑定摘要 OID、Memory ref OID 和投影水位	SQLite FTS5 不需要独立服务；投影可以从对象树重建
分层召回	先通过 Intent/Task/Run 关系、代码版本、时间和路径缩小候选，再使用 FTS5/BM25 排序摘要；Agent 只对 Top K 摘要按需展开原始证据	结构化约束避免错误版本污染，关键词基线保证无 embedding 也能工作
上下文注入	将选中的短摘要适配到记忆锚点（MemoryAnchor）；上下文选择凭证（ContextReceipt）记录摘要版本、来源、代码状态和 token 使用	现有 MemoryAnchor 与 ContextFrame 提供接入位置

版本关系

查询时同时保存“记忆来自哪个代码状态”和“当前正在查看哪个代码状态”：

关系	含义	Agent 的使用方式
Exact	当前代码 OID 与摘要结果 OID 一致	作为当前版本的直接研发上下文
Ancestor	摘要来自当前代码历史中的祖先版本	结合相关路径变化和后续 Evidence 判断适用范围
Revalidated	代码已经变化，相关测试在当前版本重新通过	附当前验证证据后使用
Diverged	摘要来自另一条分支	用于方案对比和分支演进说明
workspace	Run 产生 PatchSet 或工作区快照，尚未形成 Commit	保留为一次完整尝试并关联后续 Run
Future	摘要版本晚于查询时间点	从时间回溯结果中排除，保证评测和重放一致

时间戳负责排序，Git DAG 与 PatchSet 负责版本关系，Evidence 负责说明观察结果。Agent 在这些结构化信息上完成相关性和根因判断。

轻量化存储方案

持久化内容包括：

```
refs/libra/memory/process
├── tree
│   ├── manifest.json
│   ├── tasks/<task-id>.md
│   └── intents/<intent-id>.md
```

`manifest.json` 保存摘要 ID、Intent/Task 关系、代码 OID、相关路径、更新时间和内容摘要，可直接完成第一轮筛选。Markdown 正文供 Agent 与开发者阅读，frontmatter 供程序校验和过滤。Git 历史保存旧修订，当前 tree 表示最新摘要视图。

上述对象树和内部引用是 Memory 的权威历史。v1 同时在现有 `.libra/libra.db` 中增加一组最小、可重建的检索投影：普通投影记录摘要 OID、Memory ref OID、Intent/Task ID、代码 OID、相关路径、有效时间、内容摘要和投影水位；FTS5 虚拟表只索引 Task/Intent 摘要的检索文本；BM25 只对已经通过范围与版本过滤的候选进行排序。原始 Session 和 Evidence 不复制为新的 Memory 事实，需要时仍沿来源引用读取。

FTS5 投影不参与摘要 OID、记忆版本或事件真相的计算。投影丢失或水位落后时，系统根据受保护 Memory ref 指向的对象树重建；重建完成前不得把过期结果注入 Agent。实现通过正式 SQLite migration 加入现有数据库，不引入新的数据库文件、向量库或后台检索服务。

初版明确采用 FTS5/BM25，但分词策略仍需在 P0 冻结。unicode61 对英文与代码符号有效，却不能可靠切分连续中文；实现应为中文需求、中文 Session 和代码标识符生成规范化检索文本，并以固定中英混合语料验证。FTS5 不可用时，系统降级为关系、版本和 manifest 的确定性读取，同时由 doctor/status 报告能力缺失。

可行性

能力	Libra 当前基础	本项目实现内容
研发事实捕获	Intent、Task、Run、Session、Evidence、Decision、PatchSet 已存在	补齐稳定关联和事件覆盖范围
代码版本绑定	Git-compatible object store、Commit/Tree/PatchSet 已存在	统一 base/result/workspace OID 记录
版本化摘要	对象写入、tree、内部 ref、CAS 模式已存在	增加摘要 schema、记忆提交与恢复流程
自动编译	Agent provider/runtime 已存在	增加后台任务、幂等键、重试与校验
轻量关键词召回	现有仓库数据库使用 SQLite，但尚未建立 FTS5 表或 BM25 查询	增加可重建投影、分词规范、投影水位和重建器
Prompt 接入	MemoryAnchor、ContextFrame 和 token budget 已存在	增加版本化摘要适配和选择凭证
Benchmark	Git 历史、Session 和 Run 关系可提供样本	生成问题集、基线、指标与结果产物

项目实现细节梳理

1. 研发回合与需求迭代

研发回合以一个 Task 为根，保留其全部 Run：

Intent Revision

- └─ Task
 - └─ Run 1: 方案 A, 测试失败
 - └─ Session / ToolInvocation
 - └─ Evidence / Decision
 - └─ PatchSet / workspace OID
 - └─ Run 2: 定位根因, 修改方案
 - └─ Run 3: 完成实现, 形成结果 Commit

Task 摘要说明目标、约束、每次尝试、代码变化、观察结果、Agent 推断和未解决事项。Intent 摘要说明需求修订、Task 分解、设计变化和代码版本演进。Intent 摘要引用 Task 摘要，Task 摘要引用原始对象，形成两级导航。

2. 摘要数据契约

任务摘要采用以下最小结构：

```
---
schema: libra.task_episode_summary.v1
task_id: "..."
intent_id: "..."
run_ids: ["..."]
base_code_oid: "..."
result_code_oid: null          # 无 Commit 时使用 PatchSet / workspace OID
patchset_ids: ["..."]
related_paths: ["src/internal/ai/..."]
evidence_refs: ["..."]
decision_refs: ["..."]
session_ranges: ["..."]
covered_event_range: { from: 120, to: 187 }
source_digest: "..."
generator:
  provider: "..."
  model: "..."
  prompt_version: 1
  policy_version: 1
---
```

目标与约束
执行尝试
已观察事实
Agent 推断
代码变化

未解决事项

主要结论同时保存 `claim_kind: observed | inferred` 和来源引用。摘要修订生成新的对象和记忆版本提交, `source_digest` 与事件范围提供幂等判断。原始事实可以确定性重建; 大模型摘要通过生成器信息追踪生成条件, 并以来源一致、结构等价作为重建目标。

3. 自动编译流程

生命周期信号	系统动作
Run 进入终态	将关联 Task 标记为待编译, 记录新增 Run 与事件范围
新 Run 开始	延续同一个 Task 的研发回合, 保留之前的尝试
Task 进入终态	生成稳定的 Task Episode Summary
Intent 产生修订或进入终态	聚合相关 Task, 生成 Intent Iteration Summary
代码版本提交完成	将 Commit/Tree OID 写入对应 Run 或 PatchSet 的版本关系

“任务结束”对应 Libra Task 的终态事件, 代码提交负责提供版本坐标。无 Commit 的 Run 使用 PatchSet 或工作区快照记录代码状态。

编译器按以下顺序执行:

读取 Task 关系与新增事件

→ 按显式引用收集 Session / Evidence / Decision / PatchSet

→ 在模型调用和对象写入前完成脱敏

→ 组装 observed facts 与 inferred claims

→ 摘要 Agent 生成固定结构

→ 校验 schema、来源引用、事件范围和代码 OID

→ 写入摘要对象、tree 与记忆版本提交

→ CAS 更新受保护的 Memory ref

编译任务使用 `task_id + covered_event_range + source_digest + prompt_version` 生成幂等键。运行失败时保留待编译标记, 下一次后台调度继续处理。并发编译通过 expected ref OID 和 CAS 合并到同一条记忆历史。

脱敏沿用 Libra 现有 Redactor, 并增加 Memory 场景规则: API key、token、私钥和凭据替换为类型化占位符; 主目录、临时目录和内部服务器地址按策略规范化; 脱敏策略版本写入生成记录。摘要只接收脱敏后的结构化输入。

4. 召回与上下文组装

Agent 问题	默认读取路径
当前代码如何工作	当前 tree、当前文档和运行投影
哪次任务修改了这里	路径与版本过滤 → FTS5/BM25 → Task Summary → Run / PatchSet
为什么采用当前设计	Intent/Task 范围过滤 → FTS5/BM25 → Summary → Decision 与 成功/失败 Run
某次失败与当前问题是否相似	版本与路径过滤 → FTS5/BM25 → 失败 Run 的现象与 Evidence
一个需求经历了哪些变化	Intent 范围 → FTS5/BM25 → Task Summaries → code OIDs

初版召回器先用 Intent、Task、路径、代码关系和时间范围缩小候选，再对候选摘要执行 SQLite FTS5 MATCH，使用 BM25 相关性分数确定 Top K。BM25 只回答“关键词上哪份摘要更相关”，不能覆盖代码版本适用性或权限判断。需要确认具体原因时，Agent 再沿显式引用展开 Evidence、Session 片段或 diff，读取成本与问题复杂度对应。

查询规范化会限制关键词数量、转义 FTS 语法并对中英文与代码标识符执行固定分词；查询始终使用参数绑定。无 FTS 命中时不返回任意“最近消息”，而是退回当前 Intent/Task 范围内的 manifest 与版本排序结果。Top K 同时受对象数、字节数和 token 预算约束。

选中的短摘要通过现有记忆锚点（MemoryAnchor）进入 prompt。上下文选择凭证（ContextReceipt）记录查询时的代码 OID、Memory ref OID、FTS schema/tokenizer 版本、投影水位、BM25 排序结果、选中摘要、展开来源和 token 使用，支持相同时间点的重放与 Benchmark。

5. 内部模块

模块	职责	主要输入与输出
EpisodeAssembler	按 Task 组装完整研发回合	task_id → DevelopmentEpisode
ProcessMemoryCompiler	脱敏、结构化、调用摘要 Agent、生成编译记录	DevelopmentEpisode → TaskEpisodeSummary
VersionedMemoryStore	写对象树、创建记忆提交、CAS 更新内部引用	Summary + expected_ref_oid → committed revision
IntentSummaryCompiler	增量聚合 Task 摘要	intent_id → IntentIterationSummary
MemorySearchProjection	从 Memory ref 构建或重建 SQLite FTS5 投影，维护 schema/tokenizer 版本与水位	memory_ref_oid → indexed summaries + watermark
ProcessMemoryResolver	按关系、代码版本和预算过滤候选，使用 BM25 排序并按需展开内容	query + frozen context → resolved selection
MemoryContextAdapter	生成 MemoryAnchor 与 ContextReceipt	resolved selection → prompt segment + receipt

权威摘要仍通过一个写入入口提交；FTS5 是写入后的派生投影，不构成第二个事实写入器。CLI、Agent hook 和未来 MCP 工具调用相同的编译、存储和解析接口。

6. Benchmark 方案

LibraMemBench 从已有 Intent / Task / Run / Session / Commit 关系生成带答案来源的样本，覆盖五类问题：

- 1. 代码来源：哪个 Task/Run 引入了某段实现；
- 2. 失败回忆：过去出现过哪些相似失败及其证据；
- 3. 设计理由：当前方案经历了哪些替代方案和决策；
- 4. 需求演进：Intent 修订如何改变 Task 与代码；
- 5. 时间一致性：在历史代码版本上查询时，只使用当时已经产生的资料。

评测比较五组配置：

- 当前 Libra 上下文，无过程记忆；
- 对原始 Session 直接运行 FTS5/BM25；
- 仅使用关系、版本与 manifest 读取两级摘要；
- 关系与版本过滤、两级摘要、FTS5/BM25 排序；
- 问题路由、FTS5/BM25、两级摘要与按需证据展开。

维度	指标
来源正确性	Task/Run/Session/Evidence/代码 OID 引用完整率与正确率
版本正确性	future leakage、错误版本引用、分支关系判断
召回质量	Recall@K、MRR、NDCG@K、无命中率，以及目标 Task/Intent 摘要的排序位置
回答质量	答案充分度、设计决策覆盖率、事实与推断标记准确率
上下文成本	输入 token、展开层数、读取对象数
系统性能	P50/P95 延迟、单次编译对象数、FTS 索引体积、存储增长和重建时间

固定时间切分、数据集版本、模型、提示词、FTS schema/tokenizer 版本和硬件环境。来源引用完整率、时间泄漏、投影重建等价性和预算遵从从第一版进入 CI；召回与延迟阈值在 leris 基线完成后写入验收合同。FTS5/BM25 是 v1 基线，Benchmark 决定是否有必要增加本地 embedding、向量重排或 Agent 多步检索，不再决定是否采用关键词检索。

项目开发规划

开发以可独立评审的纵切推进，每个阶段形成一组代码、测试和文档变更。

阶段	实现内容	交付与验收
P0: 设计合同	对齐领域关系、两级摘要 schema、记忆 ref、生命周期触发、版本关系、脱敏、FTS 投影/schema/tokenizer 和 Benchmark 合同	更新 <code>docs/development/tracing/memory.md</code> ; ADR; schema 示例; 中英与代码分词样例; mentor 完成设计评审
P1: 来源链补全	完成 Session-Run-Task-Intent 关联; 记录 base/result/workspace OID、PatchSet 和事件范围	失败、重试、取消、无 Commit Run 均能组装; 关系完整性测试通过
P2: Task 摘要纵切	实现 EpisodeAssembler、ProcessMemoryCompiler、Markdown/frontmatter 编码、对象树与 ref CAS	一个真实 Task 可自动生成摘要; 增量重编译等; 并发与重试测试通过
P3: Intent 摘要与版本历史	增量聚合多个 Task; 生成 Intent 摘要; 实现记忆 revision diff、历史读取与重建	需求迭代可沿摘要展开; 任一修订能回到来源对象和代码版本
P4: FTS5/BM25 召回与 Agent 接入	实现投影 migration、水位与重建器、查询规范化、FTS5/BM25、问题路由、版本关系、渐进展开、MemoryAnchor 和 ContextReceipt	Agent 能回答五类目标问题; 投影删除后可重建; 固定上下文 Top K 可重放; token 预算测试通过
P5: Benchmark 与检索扩展决策	建立 LibraMemBench, 运行五组基线, 分析召回质量、上下文成本和延迟	原始 JSON、复现命令和报告; 依据结果决定是否增加本地 embedding、向量重排或 Agent 多步检索
P6: 交付与发布	完善模块 README、API 文档、测试索引、CI、Benchmark 文章和使用示例	<code>cargo fmt</code> 、 <code>cargo clippy -D warnings</code> 、全部相关测试通过; 核心纯逻辑单元测试覆盖率达到 100%

首个评审里程碑覆盖 P0-P2, 用一个真实 Task 跑通以下链路:

Intent / Task / Run / Session / Evidence / PatchSet

- 自动编译 Task Episode Summary
- 创建记忆版本提交
- 更新受保护 Memory ref
- 查看摘要、来源和代码 OID

P3-P4 完成 Agent 使用闭环，P5 量化两级摘要与按需展开的效果，P6 完成项目要求的文档、测试、CI 和公开 Benchmark 文章。